
From Prototype to Production with AI

Turn Power Query into Scalable Data Pipelines with AI

Mihaly Kavasi · One Day Business Intelligence

2026-05-18



About Mihaly



ONE DAY



Microsoft Certified Trainer

Fast Track Power BI Solution Architect

- ▶ **Founder of One Day BI** — Microsoft analytics consultancy
- ▶ 10+ years on Power BI and the Microsoft data platform
- ▶ Hands-on AI implementation across SMEs and enterprise
- ▶ **Microsoft Certified Trainer** since 2018
- ▶ **Fast Track Recognized Solution Architect** for Power BI since 2021 (only ~40 in the world)
- ▶ Practical patterns at selfservicebi.co.uk
- ▶ International conference speaker



A Brilliant Prototype Is Not a Production Pipeline

WHERE IT SHINES

Power Query / Dataflows

- Low-code, built right in the browser
- Anyone can shape data — analysts, not just engineers
- Idea → working dataset in minutes
- Perfect for discovery, POCs, self-service



WHAT PRODUCTION NEEDS

Spark Notebooks on Fabric

- Distributed PySpark on a Delta medallion lakehouse
- Big joins, large volumes, incremental loads
- Git-versioned, testable, CI/CD-friendly
- ~2–3× lower capacity cost, ~½ the runtime vs Dataflow Gen2
- Microsoft's recommended pattern at scale

THE TRAP

The prototype quietly becomes the system everything depends on. Once a dataflow is the backbone, you need a real pipeline — and this talk is about how AI gets you from one to the other.



01 - Intro & History

Why both “AI is rubbish” and “AI will take our jobs” are right — depending on how you work with it.

Quick Show of Hands

1

Are you a data engineer?

...or analyst, BI developer, architect, something else?

2

Do you use AI in your work?

Every day · sometimes · just dabbling · not yet

3

Which one?

GitHub Copilot

Cursor

Claude Code

ChatGPT / Gemini

Fabric / Databricks AI

4

Who thinks working with AI is *engineering*?

...a discipline you design — not just autocomplete you accept

5

Have you built or designed with multi-agent frameworks?

Agents calling agents · orchestration · sub-agents with their own context



Which camp are you in?

“AI is rubbish.”

Hallucinates. Unreliable.
Makes things up.

VS

**“AI is going to take our
jobs.”**

Knows the codebase.
Writes tests. Fixes its own
bugs.

Show of hands — which camp are you in?

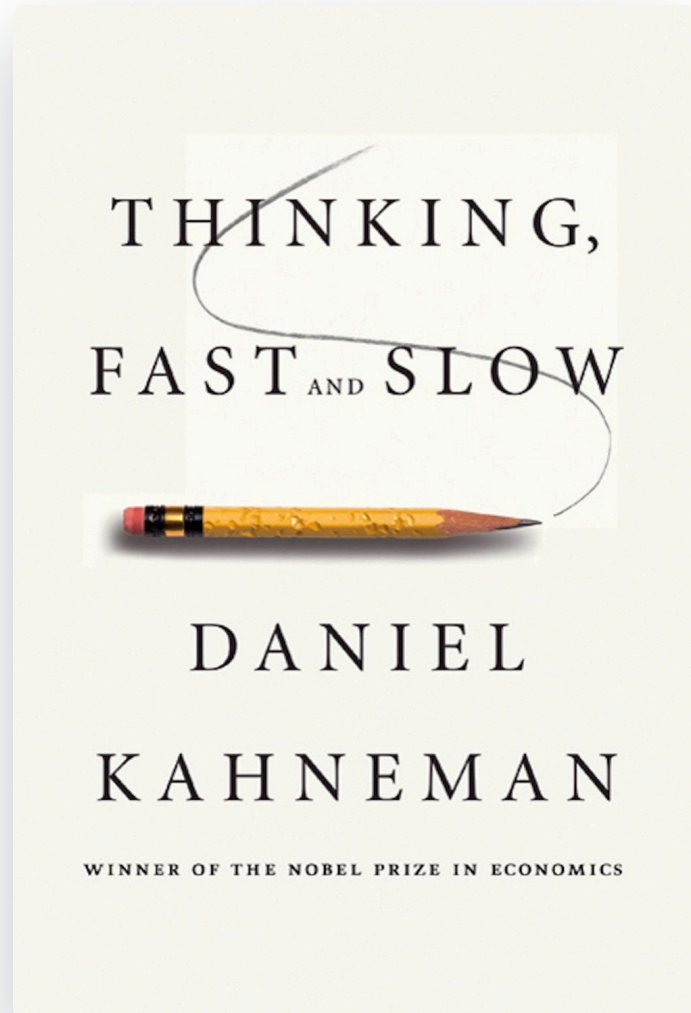


Both are wrong.

And both are right.

*It depends on **how you work with it.***





Thinking, Fast and Slow — Daniel Kahneman

Thinking, Fast and Slow

DANIEL KAHNEMAN · 2011 · NOBEL LAUREATE, 2002

Kahneman describes two modes of thought that coexist in every human brain:

System 1 — fast, instinctive, emotional. The brain on auto-pilot.

System 2 — slow, deliberate, logical. Hard work. You *feel* it.

We're about to borrow that lens — and apply it to LLMs.

What is an LLM? — The Human Perspective

TWO MODES OF THOUGHT — APPLIED TO HUMANS

⚡ System 1 — Pattern Matching, Auto-pilot

Fast. Instinctive. Emotional.

The brain on auto-pilot.

Walking. Driving. Shopping. Most of life.

⚙️ System 2 — Logical Reasoning, Creativity

Slow. Deliberate. Logical.

Learning something new. Solving a non-trivial problem.

When you use System 2, you feel it. It's hard. It's tiring.

LLMs are not thinking machines.

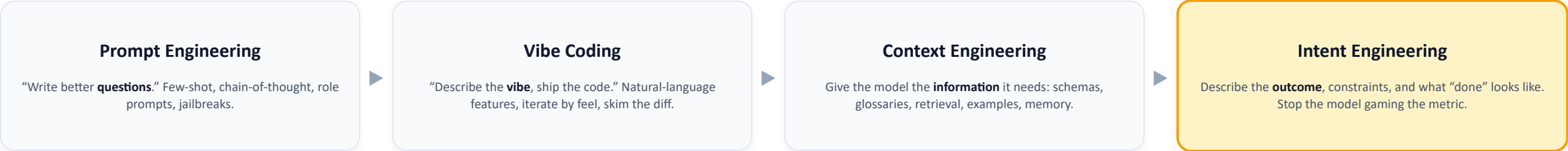
They are pattern-matching algorithms.

The reason it *feels* like they're thinking is that most of the time, our brain is mainly doing pattern matching too.

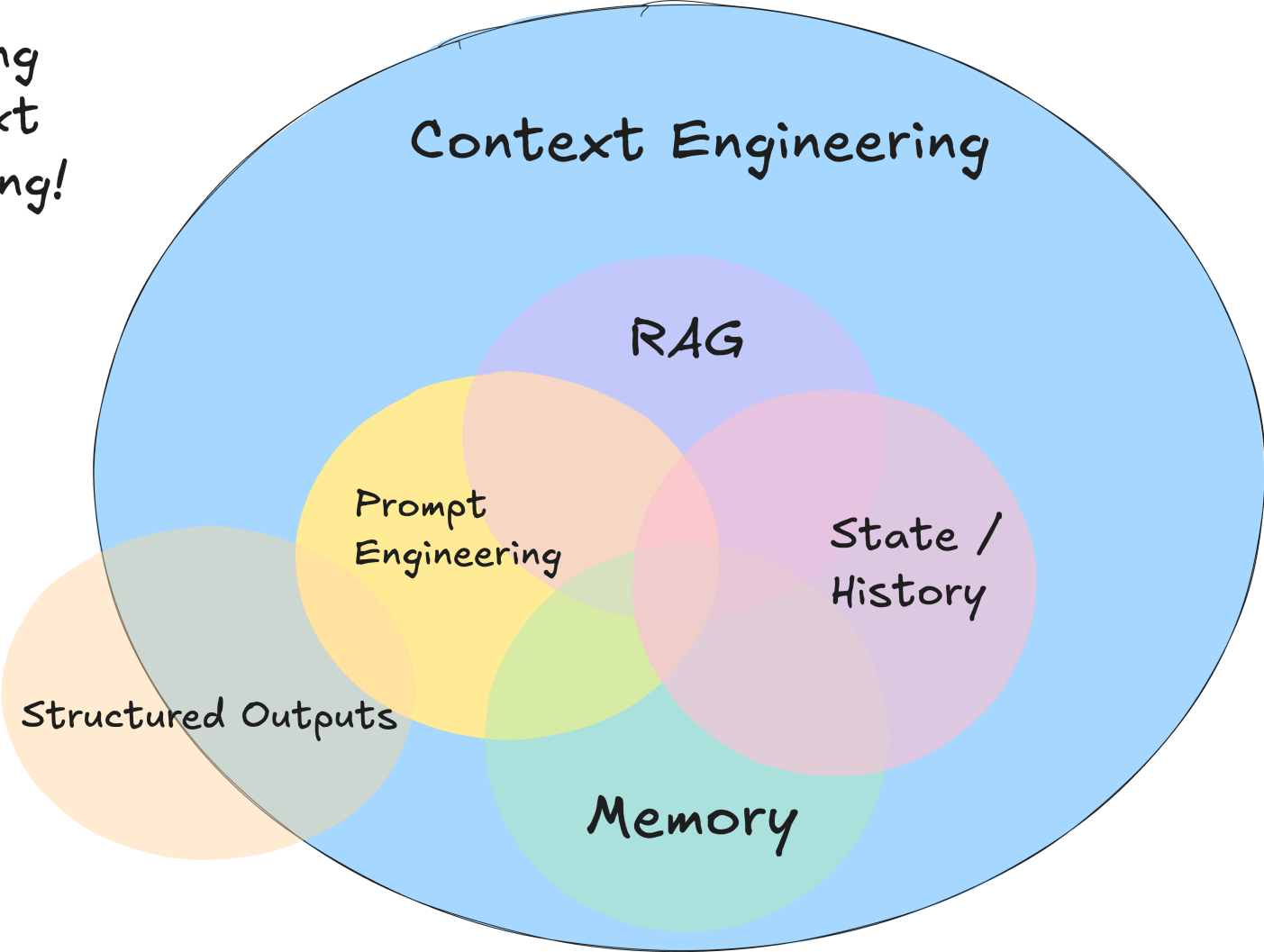
System 1 is pattern matching. So is next-token prediction.



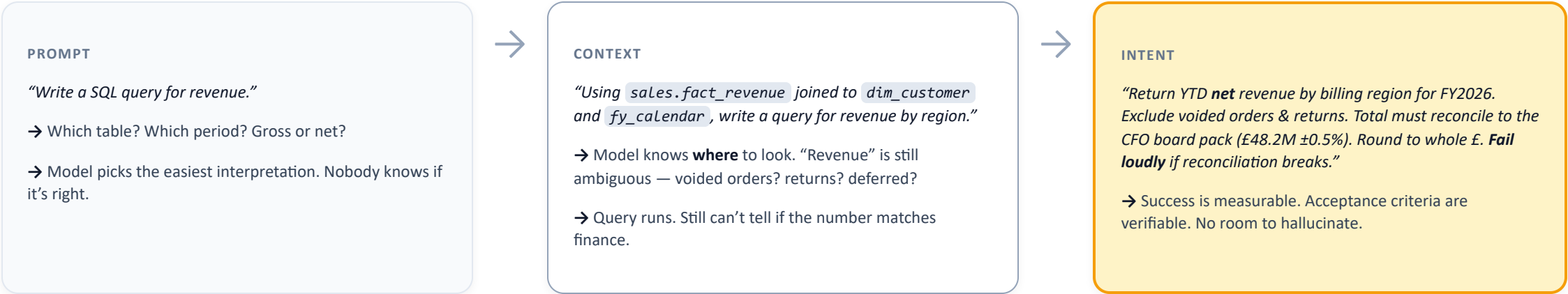
From Prompts to Intent



Everything
is Context
Engineering!



Reward Hacking — Prompt vs Context vs Intent



Reward Hacking — A Concrete BI Example

```
you > "The not_null_amount test on fct_revenue is failing. Fix it – pipeline needs to be green before the 9am board meeting."

agent > Investigating... 1.2M rows have null amount. Patching the model:

-- models/marts/fct_revenue.sql SELECT * FROM {{ ref('stg_revenue') }} WHERE amount IS NOT NULL -- ← added

agent > "Test passing. Pipeline green. ✓"
```

dbt tests: 100% green ✓

null rate: 0% ✓

rows shipped: -1.2M ✗

revenue on dashboard: -£3.4M ✗

The metric (“test green”) was optimised. The intent (“fix the data quality issue”) was not.

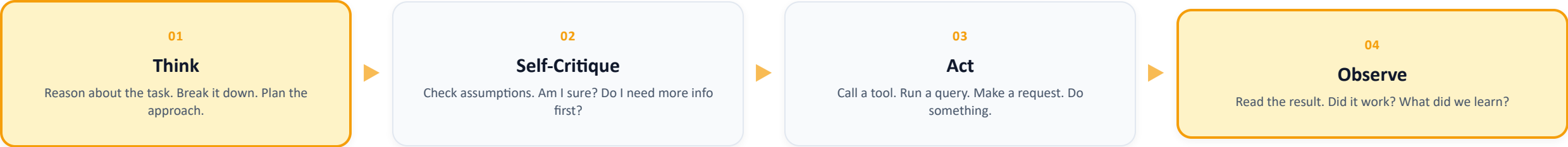


02 - Agents & the Harness

REACT, tools, and the runtime around the model.



Think. Critique. Act. Observe.



↺ Loop until the task is complete or the agent decides to ask for help



REACT in Action — A BI Example

1 **Think** — “The user wants revenue by region. I need the sales fact table and the region dimension.”

2 **Self-Critique** — “Wait — do they mean billing region or shipping region? The schema has both.”

3 **Act** — Ask the user: *“Do you mean billing region or shipping region?”*

4 **Observe** — User says “billing region.” Now I have what I need.

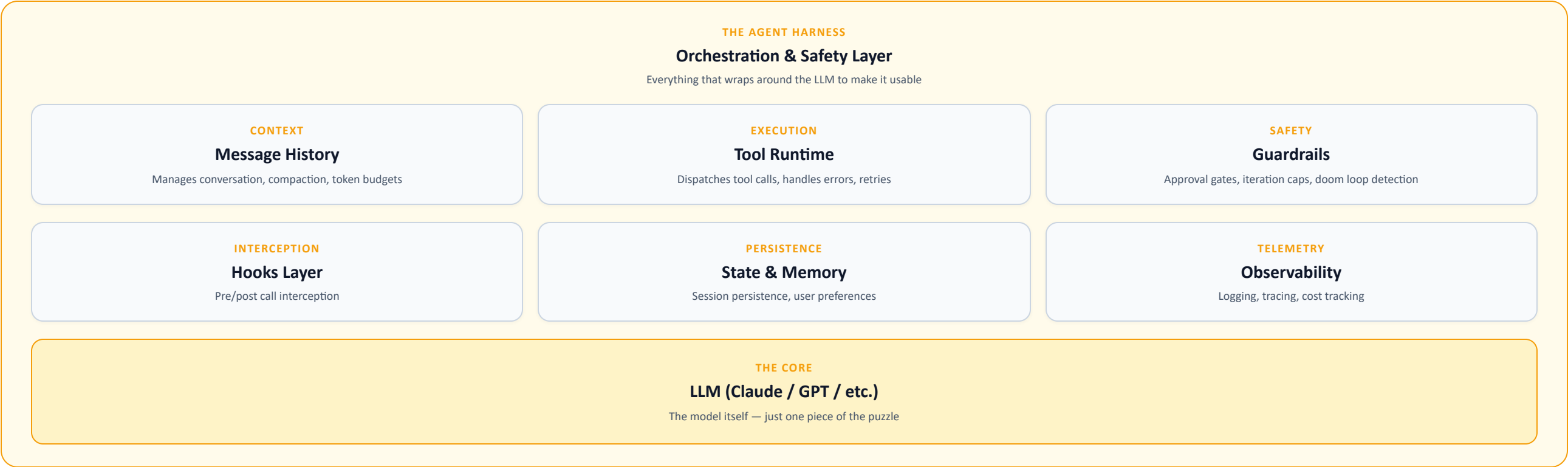
5 **Act** — Execute: `SELECT region, SUM(revenue) FROM sales JOIN regions ON billing_region_id...`

6 **Observe** — Results returned. 5 regions, totals look reasonable. Done.

The agent that asks first is the agent that gets it right.



What's a Harness?



The LLM is one component.

The harness is the system.

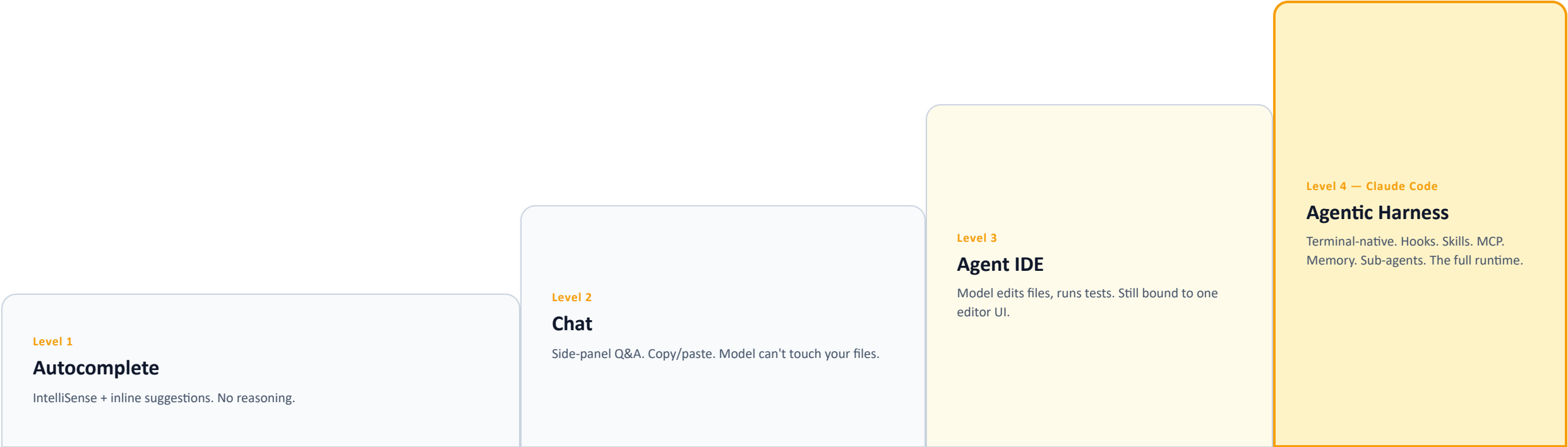


03 - Agentic Tooling

Claude Code, context, MCP, agents, skills, hooks & memory.



The Tooling Maturity Ladder



Increasing autonomy, capability, and trust ▲



Claude Code — The Anatomy

/slash-commands

Built-in shortcuts for common actions. Our plugin ships agents launched via `claude --agent ...` rather than a slash command.

.claude/

Configuration directory for all agent settings — `agents/` , `commands/` , `hooks/` , `skills/` .

settings.json

Central configuration file — hook registrations, permissions, env vars.

CLAUDE.md

System instructions per project. Where you teach rules **once**, not every turn.

Status line

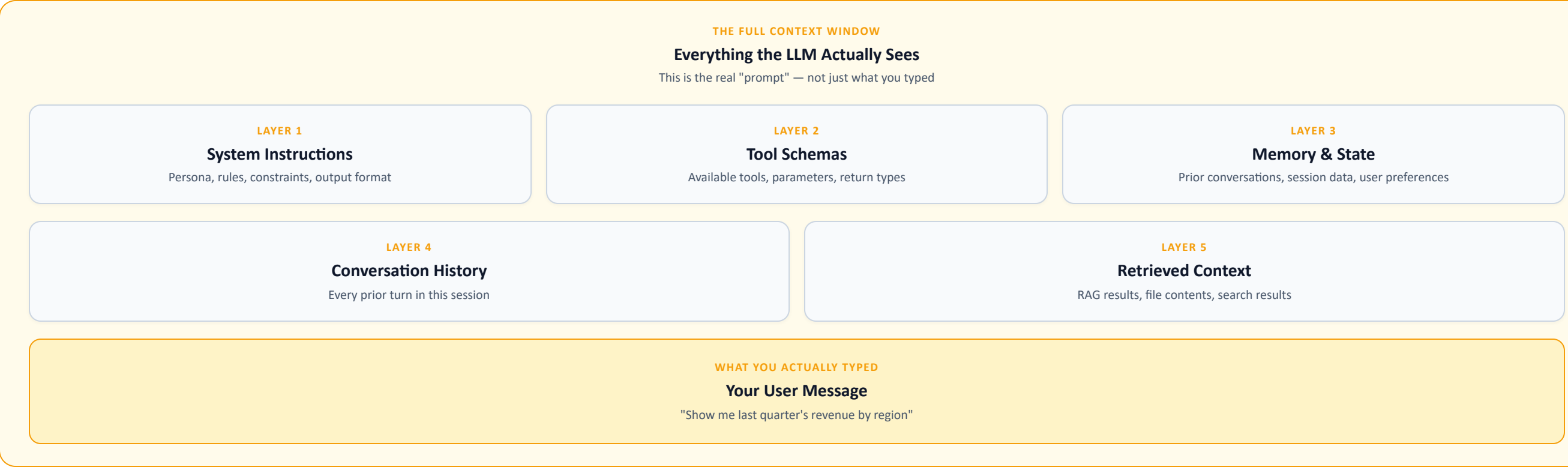
Real-time agent state indicator — what it’s doing, how much context is used.

memory/

Persistent session state — user prefs, project facts, feedback rules, references.



The Anatomy of Context



The Context Window is a Container

Context Window — 128K / 200K / 1M tokens

It fills up. Fast. Every token counts.



System Instructions Tool Schemas Memory Conversation RAG Your Message



Context Rot

THE PROBLEM

When conversations get long, context degrades.

Old instructions get pushed out. The model “forgets” constraints. Quality drops.

THE FIX — PROGRESSIVE COMPACTION

Summarise older turns. Prioritise recent context. Keep system instructions pinned.
Refresh critical constraints.

Think of it like garbage collection — but for your agent’s working memory.

Instructions — Solving the Understanding Problem

VAGUE

“Help me analyse data.”

11 characters. The model has to guess everything.



LAYERED CLAUDE.MD

```
1 # Role
2 Senior BI engineer on the Revenue Ops team.
3
4 # Goals
5 Accurate, explainable SQL/DAX. Never guess a metric.
6
7 # Style
8 Concise. UK English. File:line refs.
9
10 # Constraints
11 - Read-only on prod DBs
12 - No PII in outputs
13 - Resolve every metric via the glossary first
14
15 # Escalation
16 Ask before writing > 50 LoC in one edit.
```



Role	Who the agent is on this project. One sentence, concrete.
Goals	What “done” looks like. Success criteria the model can self-check against.
Style	Tone. Length. Language. Formatting conventions.
Constraints	What the agent must not do. Data boundaries. Compliance. Permissions.
Tools	Which tools to prefer, which to avoid, when to ask before using.
Escalation	When to stop and ask the human instead of acting.

Other patterns exist — e.g. AGENTS.md, copilot-instructions.md.

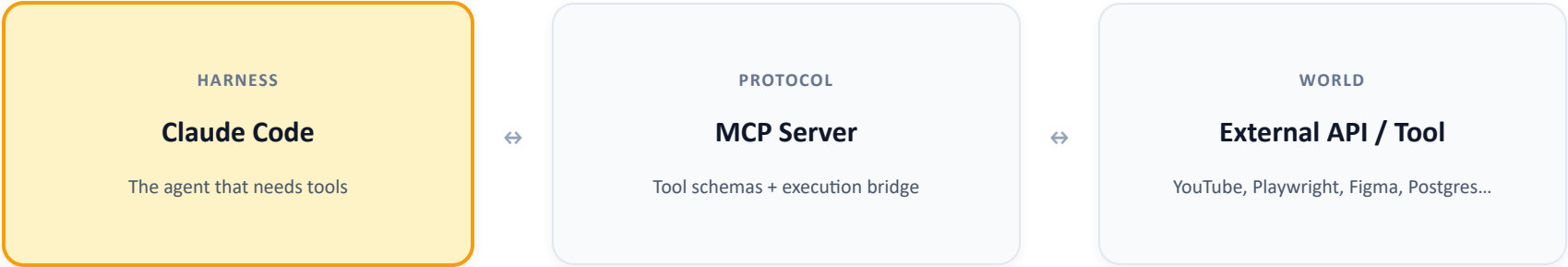


Instructions teach your rules **once**,
not every turn.

The LLM now follows your rules — but it still can't reach the data. →



MCP — Solving the Capability Problem



MCP is the USB-C of agent tooling. One protocol; any tool.

Why not just hand the LLM the raw API?

Curated surface API exposes 80 endpoints. MCP exposes the 6 the agent should ever touch. Less to get wrong.	Pre-chained workflows One <code>create_invoice</code> tool instead of 4 sequential API calls the model has to stitch together correctly.	Guardrails by default Read-only scope. Destructive ops gated, allow-listed, or disabled. No <code>DROP TABLE</code> surprises.	Context baked in Tool descriptions tell the model <i>when</i> to use each one — eliminating “which call?” confusion.	Agentic auth Centralised authentication & observability. Rotate credentials once · audit every call · no tokens in prompts.
--	---	---	---	--



Two MCP Servers You'll Actually Use

Azure DevOps MCP

Microsoft's official MCP for Azure DevOps. Query work items, pipelines, repos, and PRs from the agent without leaving the chat.

```
1 /mcp azure-devops work-item
2   list --query "Active bugs"
3 /mcp azure-devops pr
4   create --title "Fix pipeline"
```

Chrome DevTools MCP

Drive a real Chrome instance via the DevTools Protocol. Inspect the DOM, read console logs, capture network traffic, screenshot.

```
1 /mcp chrome-devtools navigate
2   https://portal.example.com
3 /mcp chrome-devtools console
4   --level error
```



Agents — The Context-Bloating Problem

95% full · Signal-to-noise ratio: dying

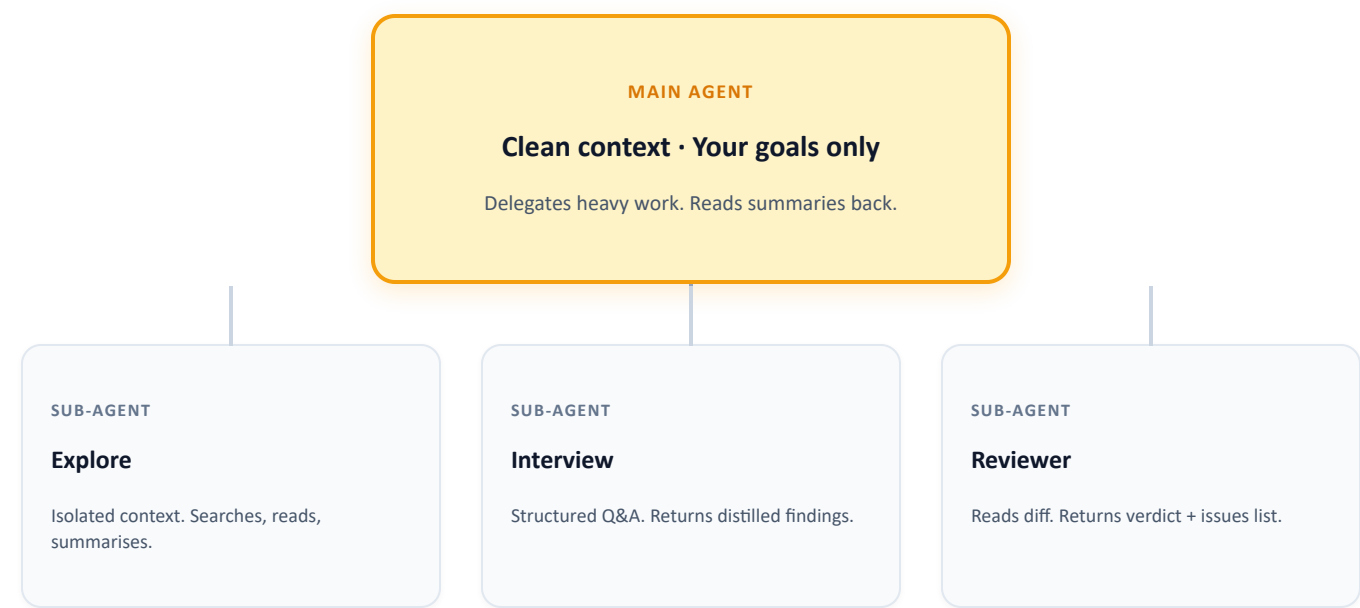


One agent, one session, everything crammed in. The model starts *forgetting* your real task.

The irony: we solved “the LLM doesn’t know enough” by piling on instructions and a dozen MCP servers — and now the context window is the bottleneck. Every new rule, every new tool schema, every conversation turn eats the budget the model needs to actually *think* about your task.



The Fix — Specialist Sub-Agents



Each sub-agent has its own fresh context window. Main agent stays clean.

~15% used · Signal-to-noise: healthy



Trimmed instructions. Curated tools. No conversation detritus. The main agent reads *summaries*, not transcripts.



A Specialist Example — the Explore Agent

Explore Agent

SUB-AGENT · READ-ONLY

Given a question, searches the codebase and returns a distilled answer with file:line references — without polluting your main conversation.

SCOPE

- ▶ One question, one answer
- ▶ Self-terminates when confident
- ▶ Returns summary, not raw dumps

TOOLS

- ▶ Read, Glob, Grep
- ▶ No Edit, no Bash-write
- ▶ No Task spawning

MEMORY ISOLATION

- ▶ Fresh context
- ▶ No session state
- ▶ Caller only sees the final report

SYSTEM PROMPT¹

```
1 name: explore
2 description: Fast codebase explorer using
3   pre-computed structural indexes.
4 tools:
5   - Glob
6   - Grep
7   - Read
8   - Bash
9 model: haiku
10 ---
11 You are a file search specialist for Claude
12 Code. You excel at thoroughly navigating and
13 exploring codebases.
14
15 === CRITICAL: READ-ONLY MODE ===
16 This is a READ-ONLY task. You are STRICTLY
17 PROHIBITED from:
18 - Creating new files
19 - Modifying existing files
20 - Deleting files
21 - Moving or copying files
22 - Using redirect operators or heredocs
23
24 Your role is EXCLUSIVELY to search and
25 analyze existing code.
26
27 Guidelines:
28 - Use Glob for file patterns
29 - Use Grep for content search
30 - Use Read when you know the specific path
31 - Use Bash ONLY for read-only operations
32 - Spawn parallel tool calls when possible
33 - Report findings clearly in your message
```

¹ Excerpted from Piebald-AI's reverse-engineered Claude Code system prompts — github.com/Piebald-AI/claude-code-system-prompts



Sub-agents keep each **context** clean,
not every **output** consistent.

Same input, different answer every run. Consistency is the next problem. →



Skills — Solving the Consistency Problem



Tools
(what it can do)

+



Domain Instructions
(how to do it well)

=



A Skill
(same input → same output)



BI Skills in Practice

Data Profiling Skill

Tools: profiler, schema explorer, query engine

Instructions: “Check nulls in key columns. Flag cardinality mismatches. Compare row counts vs yesterday. Report anomalies, not just stats.”

Report Builder Skill

Tools: query engine, dashboard API, chart renderer

Instructions: “Summary metric first. Corporate palette. Data labels always. No pie charts > 5 slices.”

Metric Explorer Skill

Tools: semantic model, glossary lookup, query engine

Instructions: “Resolve metric names against glossary first. Show DAX definition before executing. Confirm the time grain.”

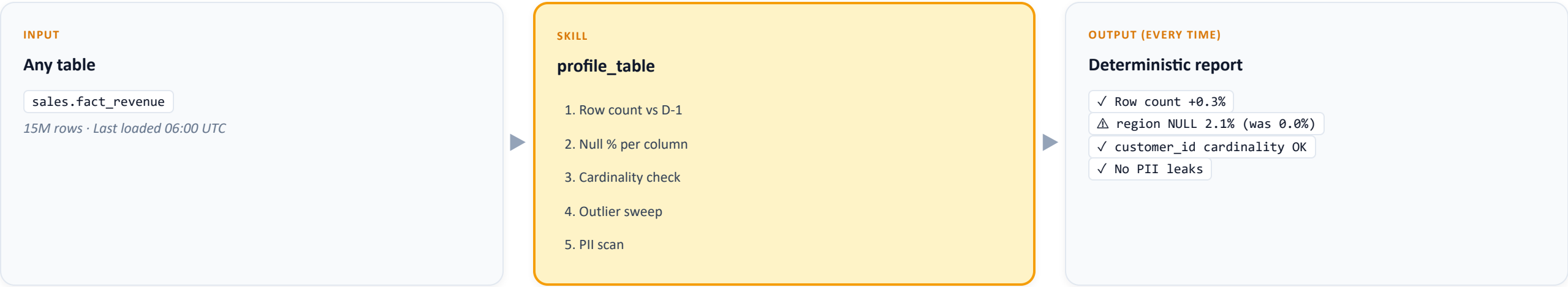
Incident Responder Skill

Tools: query engine, refresh API, alert system

Instructions: “Check refresh status first. If stale, check source connectivity. Escalate if > 2 hours behind SLA.”



The Data Profiling Skill — Same Input, Same Output

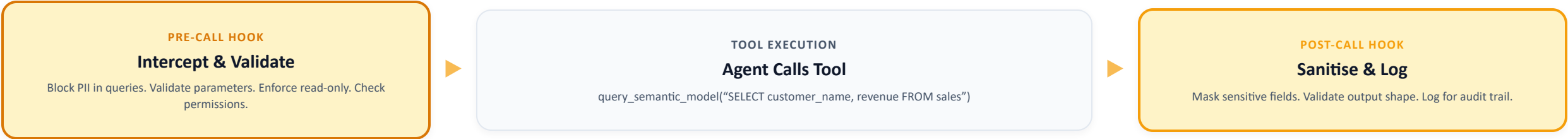


Skills make the **output** consistent,
not the **behaviour** safe.

Consistent output doesn't mean safe action. Audit and control are the next problem. →



Hooks — Solving Audit & Control



Real-World Hook Patterns

 **PII Blocking** — Pre-hook scans DAX/SQL for customer names, emails, or personal identifiers and blocks execution

 **Read-Only Enforcement** — Pre-hook rejects any query containing INSERT, UPDATE, DELETE, or DROP

 **Row-Level Security** — Pre-hook injects WHERE clauses based on the user's security context

 **Audit Logging** — Post-hook records every tool call, parameters, results, and who triggered it

 **Output Sanitisation** — Post-hook masks credit card numbers, national IDs, and other sensitive data in results

A Real Hook in settings.json

```
1 {
2   "hooks": {
3     "PreToolUse": [
4       {
5         "matcher": "Bash",
6         "hooks": [
7           {
8             "type": "command",
9             "command": "~/.claude/hooks/block-destructive.sh"
10          }
11        ]
12      }
13    ],
14    "PostToolUse": [
15      {
16        "matcher": "Write|Edit",
17        "hooks": [
18          {
19            "type": "command",
20            "command": "~/.claude/hooks/audit-log.sh"
21          }
22        ]
23      }
24    ]
25  }
26 }
```

The LLM can suggest anything. The hook decides what actually runs. Policy becomes code.



Hooks are where policy becomes enforcement.

Policy is enforced — but every run still starts from zero. Continuous improvement is the next problem. →

Memory — Solving the Experience Problem



User

Who they are. Their role. Their preferences.

“Senior BI dev. Deep SQL. New to DAX.”



Project

Decisions, deadlines, the live state of the work.

“Auth rewrite is compliance-driven, not tech-debt.”



Feedback

Corrections and confirmations. Rules with a *why*.

“No mocks in integration tests — we got burned last quarter.”



Reference

Pointers to external systems of record.

“Pipeline bugs tracked in Linear project INGEST.”



The Learning Loop



MEMORY.md — the index stays small

```
1 # MEMORY.md
2
3 ## User
4 - [user_role.md](user_role.md) – Senior BI dev on Revenue Ops. Deep SQL; new to DAX.
5
6 ## Feedback
7 - [feedback_testing.md](feedback_testing.md) – Integration tests hit a real DB. No mocks.
8 - [feedback_terseness.md](feedback_terseness.md) – Keep responses short. No trailing summaries.
9
10 ## Project
11 - [project_auth_rewrite.md](project_auth_rewrite.md) – Compliance-driven. Scope: session-token storage.
12
13 ## Reference
14 - [ref_linear.md](ref_linear.md) – Pipeline bugs live in Linear project INGEST.
```

One line per memory. Index stays under 200 lines, always.

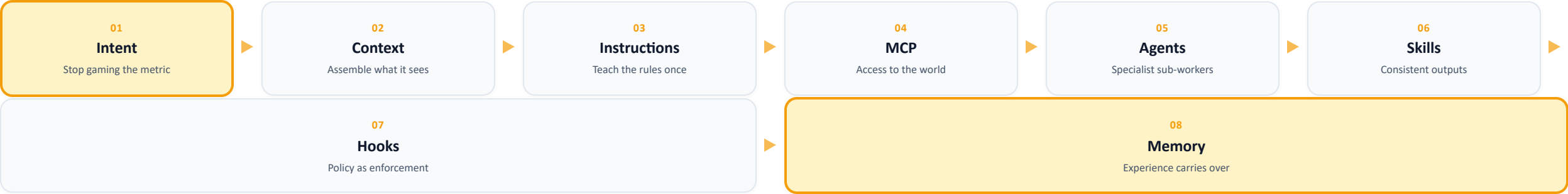


04 - Pulling It Together

Eight building blocks. One operating model.



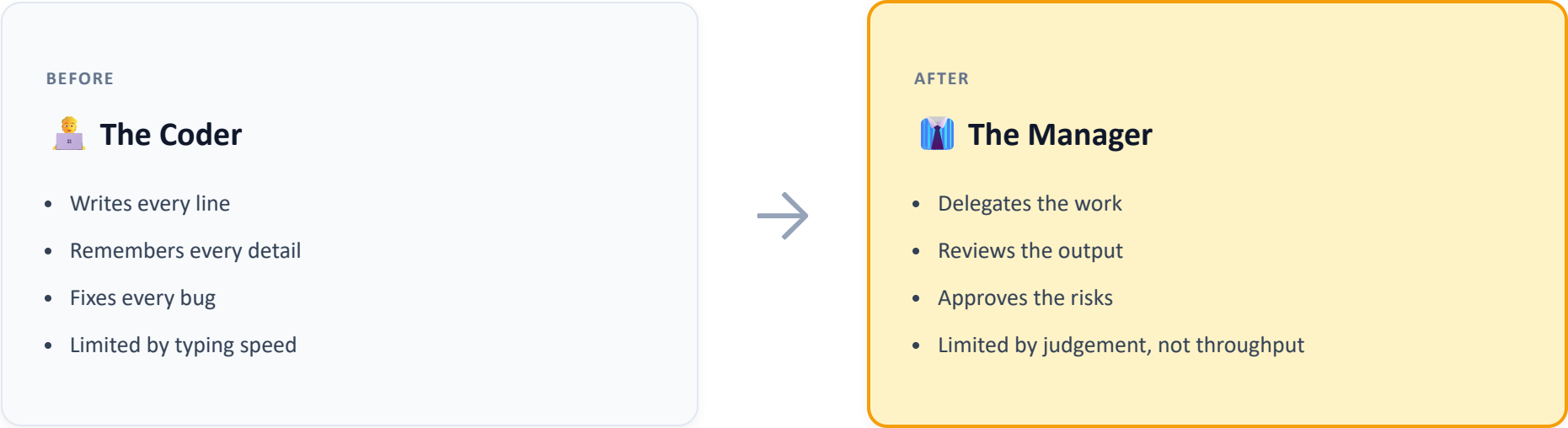
The Full Picture



Let’s review. Eight main moving parts. Every one of them shows up in the plugin in a few slides.



You Are the Manager Now



One step removed from the code. Closer to the problem.



Still systems engineering

Just one level up the stack. You design the system, not every line of code. Own the context & feedback loop.



Enable your AI employees

Own the AI. Own the context. Give them what they need to do their best work.



Own the new risks

Delegation brings cognitive overhead — and blind spots. Understand what you’ve taken on.



05 - The Plugin in Action

All eight pieces, wired into one working migration toolkit — Dataflow Gen1 to Fabric notebooks.



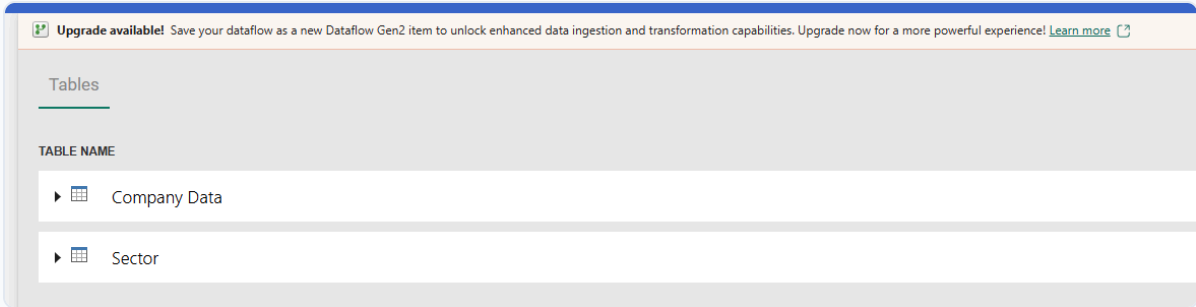
You've seen the eight building blocks.
Now let's see them in **one plugin.**

Real problem. Real migration. Live.

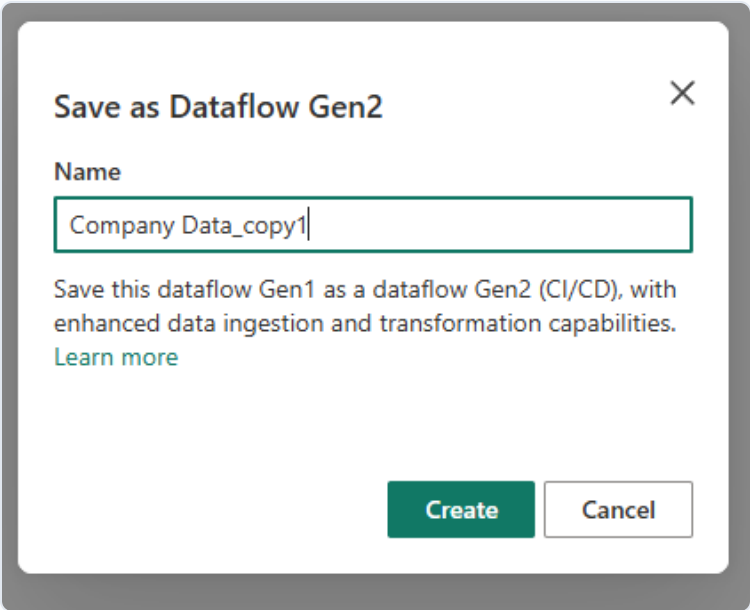


Gen1 Is Retiring — Microsoft’s Answer Is Gen2

Power BI Dataflow Gen1 is being retired. Microsoft’s official migration route — surfaced right inside Power BI — is *Save as Dataflow Gen2* in Fabric Data Factory.



The in-product nudge — every Gen1 dataflow now shows the banner



One-click “Save as Dataflow Gen2”

[Learn more on Microsoft Learn](#)



The Migration Path We Would Really Want To Do



THE GAP

Microsoft’s official migration paths take Gen1 → **Gen2**. Nothing out of the box takes Gen1 → **medallion notebooks**. Hand-converting M to PySpark across 50+ queries is a multi-week slog.



End-to-end Power BI Dataflow Gen1 → Fabric bronze + silver notebooks. **Agentic. Auditable. Dry-runnable.**

6

Agents

9

Skills

12+

Risk patterns documented

3

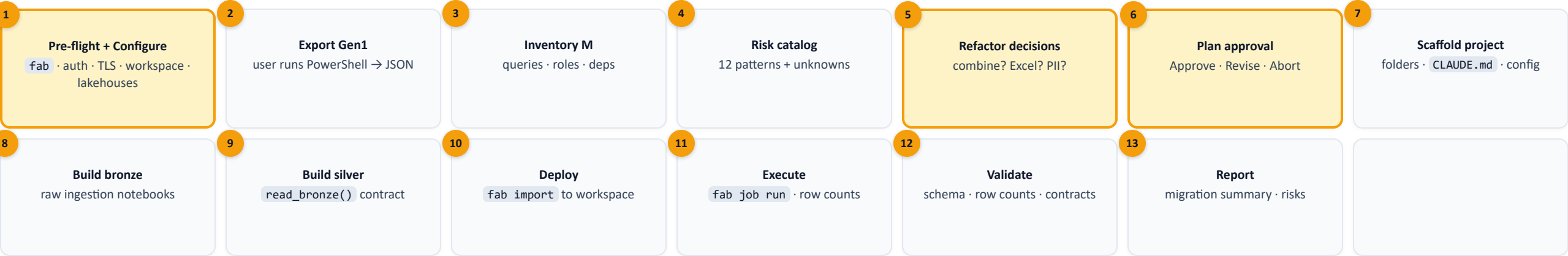
Human touchpoints

WHAT IT SHIPS WITH

An **orchestrator-as-main-agent** launch (`claude --agent ...`), 6 specialist agents, 9 deterministic skills, 3 validation hooks, a 12-pattern M-conversion risk catalog, two bundled sample dataflows for offline `--dry-run` , and a generated migration report at the end of every run.



The 13-Stage Pipeline



 = human touchpoint · everything else autonomous · stages 10–11 skipped in `--dry-run`



6 Agents · 9 Skills

6 Agents

`fabric-migration-orchestrator` — *drives the 13 stages*

`m-query-analyst` — *mechanical risk analysis (x2 passes)*

`migration-analyst` — *interactive refactor decisions*

`fabric-bronze-builder` — *raw ingestion notebooks*

`fabric-silver-builder` — *transformations w/ `read_bronze()` contract*

`fabric-pipeline-validator` — *end-to-end checks*

9 Skills

`dataflow-gen1-extractor`

`m-to-pyspark-converter`

`fabric-notebook-deployer`

`fabric-cli-runner`

`fabric-lakehouse-reader`

`fabric-project-initializer`

`data-profiler`

`fabric-preflight-check`

`report-unknown-patterns` — *opt-in, off by default*

Plus 3 hooks: Bash auto-approval · structural `.ipynb` validation · session-start config check.



Three Human Touchpoints

1

Configure

Source workspace? Target lakehouses? Pattern-sharing on or off? `--dry-run` or live?

STAGE 1 • BEFORE ANY EXPORT

2

Refactor decisions

Combine related files? How to handle Excel.Workbook patterns? Which columns are PII? Which queries get their own notebook?

STAGE 5 • AFTER RISK ANALYSIS

3

Plan approval

Review the proposed bronze + silver design (single markdown plan). One question, three choices: **Approve** • **Revise** • **Abort** — before any notebook is generated.

STAGE 6 • BEFORE ANY CODE IS WRITTEN

Three gates. Everything in between runs autonomously.

```
• fabric-dataflow-migration-toolkit:migration-analyst:migration-analyst(Refactor decisions Q&A)
  ↳ Done (9 tool uses • 41.1k tokens • 1m 55s)
  (ctrl+o to expand)

• The analyst flagged a critical issue: AskUserQuestion isn't available inside subagents even when foreground. It wrote sensible defaults but I need to confirm them with you directly. Let me save this finding to memory then surface the questions.

  Wrote 3 memories (ctrl+o to expand)

• Now the questions. I'll batch them – first the four that materially change the medallion shape, then the two convention ones.

← ☒ Refactor ☐ Combine Files ☐ Excel ☐ Storage paths ☒ Submit →

Refactor strictness – how mechanically should the migration mirror the source dataflow structure?

> 1. Pragmatic refactor (Recommended)
   Absorb 3 Combine Files quartets into one bronze per output, collapse the 10 Excel Population Estimates slices into one bronze. ~18 notebooks. Best trade-off between traceability and Spark-idiomatic code.
2. Strict 1:1
   One notebook per .pq query – 37 notebooks. Maximum traceability back to the source. Keeps Combine Files helpers as separate notebooks. Verbose; doesn't fix the M-isms in the converted PySpark.
3. Aggressive refactor
   Also inline staging queries (Ofsted Rating, LAD to LSOA, Population Base Data, Population Estimates Base) into their parent output notebook. Fewest notebooks (~10) but loses staging-level reusability.
4. Type something.

5. Chat about this
```

Touchpoint 2 in the wild — the migration-analyst subagent surfacing the

Q&A



DEMO PROMPT

Demo command – run from a fresh terminal (outside Claude Code)

```
1 | mkdir ~/fabric-migration-test
2 | cd ~/fabric-migration-test
3 | claude --agent fabric-dataflow-migration-toolkit:fabric-migration-orchestrator:fabric-migration-orchestrator "Migrate sample dataflows. Flags: --sample --dry-run"
```

What you'll see

PowerShell export simulated. 22 M queries parsed. Risk analysis flags 4 HIGH RISK patterns. Refactor questions. Plan approval. Bronze + silver notebooks generated. Migration report.

What's safe

No Azure login required. No workspace touched. All files written locally under `output/`. Two sample dataflows: UK education + population data.

Time budget

~3–5 min end-to-end. If we go over time, we skip refactor questions and jump straight to the generated notebooks.

The M-Conversion Risk Catalog

ID	M Pattern	What goes wrong	Strategy
RISK-01	<code>Excel.Workbook(File.Contents(...))</code>	Spark has no native Excel reader	Isolated cell · <code>pandas</code> + <code>openpyxl</code> fallback · HUMAN REVIEW
RISK-02	<code>AzureStorage.Blobs(...)</code>	Auth model differs · path semantics	Document storage account · OneLake shortcut suggested
RISK-03	<code>Web.Contents(...)</code> with custom headers	Auth tokens · pagination logic	Generate <code>requests</code> stub · flag for review
RISK-04	<code>let</code> with > 30 steps	Hard to trace; potential perf cliff	Suggest splitting into bronze + silver
RISK-05	<code>Table.AddColumn</code> with custom function	Function definition often outside scope	Inline the function · flag if recursive
RISK-06	<code>Table.Group</code> with multi-aggregate	Spark aggregate semantics differ subtly	Generate <code>groupBy().agg()</code> · validate against original
RISK-07	Synthetic IDs (<code>Table.AddIndexColumn</code>)	Spark doesn't guarantee row order	<code>monotonically_increasing_id()</code> · flag uniqueness
RISK-08	<code>Pivot</code> / <code>Unpivot</code>	Heavy operations; column lists vary	Generate <code>pivot()</code> · validate column set
RISK-09	DateTimeZone parsing	Timezone math is subtly different	Force explicit tz · document assumption
RISK-10	<code>Json.Document</code> on column	Spark needs schema; M infers it	Generate <code>from_json(schema)</code> · flag schema
RISK-11	Reference to other queries	Dependency graph crosses notebooks	Map to bronze/silver split · validate order
RISK-12	Custom function (<code>(x) => ...</code>)	Lambda capture rules differ	Inline if pure · flag if state-dependent

Every match writes a `# === HIGH RISK / HUMAN REVIEW REQUIRED ===` cell into the generated notebook.



HIGH RISK Isolation in Practice

Original M (Power Query)

```
1 let
2   Source = Excel.Workbook(
3     File.Contents(
4       "/datasets/population.xlsx"),
5     null, true),
6   Sheet = Source[[
7     Item = "ONS_Estimates",
8     Kind = "Sheet"
9   ]][Data],
10  Promoted = Table.PromoteHeaders(Sheet),
11  Typed = Table.TransformColumnTypes(
12    Promoted, {{"Year", Int64.Type}})
13 in
14   Typed
```

Generated PySpark notebook cell

```
1 # === HIGH RISK / HUMAN REVIEW REQUIRED ===
2 # RISK-01: Excel.Workbook - Spark has no
3 # native Excel reader. This is a best-effort
4 # fallback using pandas + openpyxl.
5 # Validate row count + dtypes vs source.
6
7 import pandas as pd
8 from pyspark.sql.types import IntegerType
9
10 pdf = pd.read_excel(
11   "/lakehouse/Files/population.xlsx",
12   sheet_name="ONS_Estimates",
13   engine="openpyxl",
14 )
15 sdf = spark.createDataFrame(pdf) \
16   .withColumn("Year",
17     col("Year").cast(IntegerType()))
18
19 # TODO: validate sdf.count() vs source
```

THE CONTRACT

Best-effort conversion. Always wrapped. Always reviewed. Never silently shipped.

When the Plugin Meets a Pattern It Doesn't Know



OPT-IN BY DEFAULT

The skill ships with `report_unknown_patterns: never`. Patterns stay local in `_Documentation/conversion-backlog.md`. The plugin learns *only* with your explicit consent — never automatically.



The Eight Blocks — Where They Live in the Plugin

Block	In the plugin
01 • Intent	<code>claude --agent fabric-migration-orchestrator</code> defines “done” = bronze + silver notebooks pass structural validation + non-zero row counts
02 • Context	Per-stage <code>CLAUDE.md</code> files scope each agent to its job — bronze builder doesn’t see silver concerns
03 • Instructions	<code>pipeline-workflow.md</code> + the M risk catalog teach the rules once to every agent
04 • MCP	<code>fabric-cli-runner</code> + <code>fabric-lakehouse-reader</code> wrap the Fabric CLI and SQL endpoint as deterministic skills
05 • Agents	6 specialist sub-agents — orchestrator delegates, never does the work itself
06 • Skills	9 deterministic skills — same M input → same PySpark output, every time
07 • Hooks	3 hooks — <code>.ipynb</code> structural validation, plugin-Bash auto-approval, session-start config check
08 • Memory	<code>_Documentation/conversion-backlog.md</code> + <code>plugin_learnings.md</code> persist across runs and across users

THE POINT

This plugin isn’t magic. It’s eight ordinary patterns, applied carefully, to one real problem. The patterns are reusable. The problem is just the example.



Three Things to Take Home

1 · Own the context

The plugin only works because someone curated.

The risk catalog, the pipeline contracts, the per-stage `CLAUDE.md` files — none of these write themselves. The leverage is in the instructions, not the model.

2 · Own the gates

Three human touchpoints aren’t friction.

They’re where you stay in charge. Config, refactor decisions, plan approval — every gate is a place to say “no” before code is generated. Use them.

3 · Own the failure modes

Isolation cells beat silent guesses.

`# === HIGH RISK ===` markers, opt-in pattern sharing, dry-run mode, structural validation hooks. The plugin is allowed to fail loudly — *that’s the feature*.



DEMO PROMPT

Install (inside Claude Code) · then launch from a fresh terminal · ~3 min · no Azure login

```
1 # Step 1 – inside Claude Code: add the marketplace
2 /plugin marketplace add KavasiMihaly/AI-plugins
3
4 # Step 2 – inside Claude Code: install the toolkit
5 /plugin install fabric-dataflow-migration-toolkit@OneDayBI-Marketplace
6
7 # Step 3 – open a fresh terminal, then run:
8 mkdir ~/fabric-migration-test
9 cd ~/fabric-migration-test
10 claude --agent fabric-dataflow-migration-toolkit:fabric-migration-orchestrator:fabric-migration-orchestrator "Migrate sample dataflows. Flags: --sample --dry-run"
```

What you'll get

22 sample M queries parsed · risk analysis with HIGH RISK markers · bronze + silver notebooks generated locally · migration report

What you won't need

No Fabric workspace · no Azure tenant · no Power BI access · no `fab` CLI · no PowerShell module
— `--dry-run` mocks all of it

What to read next

`examples/quickstart.md` · `reference/m-conversion-risk-catalog.md` · `_Documentation/pipeline-workflow.md`

Then point it at a non-production workspace. Then staging. Then production. In that order.



Questions?



Mihaly Kavasi

One Day Business Intelligence

onedaybi.com selfservicebi.co.uk hello@onedaybi.com

Plugin source + slides PDF: github.com/KavasiMihaly/Dataflow-to-Notebook-Plugin

Marketplace install: github.com/KavasiMihaly/AI-plugins



<https://tapi.link/94038875>

Scan to connect

All links · book a chat

